

SE FINAL PROJECT SPECIFICATION

Course: Software Engineering. **Semester:** Semester 2, 2025-2026

Topic: SaigonRide - Distributed Vehicle Rental System. **Team Size:** 2-3 Students per Group

1. Background & Problem Statement

Ho Chi Minh City's transit system is rapidly modernizing with **SaigonRide**, a distributed network of public bicycles and electric scooters. The system allows users to unlock a vehicle at Station A and drop it off at Station B.

However, the platform currently struggles to serve its two distinct user bases: **Local Commuters** (who use local e-wallets and cash) and **Foreign Tourists** (who rely on international payment methods and require passport verification). The city has commissioned your software engineering team to design a unified system that manages fleet inventory, dynamic time-based pricing, and robust, multi-option payment architecture.

Educational Objectives: This project simulates a real-world software engineering contract. You will practice the complete Software Development Life Cycle (SDLC), moving from abstract client requirements to system modeling, UI/UX design, database structuring, coding, and rigorous quality assurance testing.

2. Core Business Logic & System Features

Regardless of your technology stack, your system must design for the following business rules and features:

A. The Core Rules

- **Dynamic Time & Location Pricing:** Fare is calculated based on vehicle category (e.g., Standard Bike = 500 VND/min, E-Scooter = 1,500 VND/min). If a user returns a vehicle to a "Low Inventory" station (< 20% full), a 15% discount is applied to the total fare.
- **Multi-Gateway Payment Design:** The system must structurally support multiple payment methods categorized by user type (Locals: MoMo, VNPay, Cash | Tourists: Apple Pay, PayPal, Cash). *Note: Standard tiers only need to simulate these payments in the UI/database. Tier 4 must code the live API.*
- **System Reports:** Each group must implement at least **two distinct reports** (e.g., Revenue Report by Vehicle Category, Current Station Inventory/Utilization Report).

B. Expected Core Entities (For CRUD Operations)

To successfully build this system, your database and classes will likely need to manage (Create, Read, Update, Delete) the following entities:

- **Users/Actors:** Local Commuters, Foreign Tourists, System Admins.

- **Vehicles & Categories:** E-bikes, Standard Bikes, Scooters (including their statuses: Available, In-Transit, Maintenance).
- **Stations:** Locations, capacity limits, current inventory count.
- **Transactions/Rentals:** Timestamps for start/end, calculated fares, applied discounts.

3. Technology Tiers (Choose Your Target Grade)

You must self-select a technical tier. Your chosen tier dictates your technology stack and maximum base score. Choose a tier that challenges your team but remains realistic for a 7-week timeframe.

- **Tier 1 (Max Score: 7.0)** - C# WinForms + Local SQL Database / Excel. (3-Tier Architecture)
- **Tier 1B (Max Score: 7.5)** - Core PHP (or CodeIgniter) + MySQL + **Bootstrap** (for responsive UI). (MVC Architecture)
- **Tier 2 (Max Score: 8.5)** - ASP.NET Web Forms/MVC (Database First) + SQL Server + **Bootstrap**. (MVC Architecture)
- **Tier 3 (Max Score: 9.5)** - ASP.NET MVC (Code First) + Entity Framework + **Bootstrap**. (MVC Architecture)
- **Tier 4: The Innovation Pioneer (Bonus - Required for 9.5 to 10.0)** - Successfully implement a live external API integration (e.g., VNPay Sandbox, PayPal Developer, Zalo ZNS), create an interactive, advanced Admin Dashboard for system monitoring, **AND** implement **Automation Tests** for your core workflows.

4. The 7-Week SDLC Roadmap & Deliverables

Part 1: Project Proposal & Requirements Engineering (2 Weeks)

- **Total Weight:** 20% (15% Document + 5% Presentation)
- **The "1 Use Case Per Student" Requirement:** Each student must select and take ownership of at least **one main use case**. A group of 2 will have 2 main use cases; a group of 3 will have 3 main use cases. Every main use case must ensure complete **CRUD** functionality (e.g., "Manage Vehicle Inventory" involves creating, reading, updating, and deleting vehicles).
- **Document Checklist (What to include in your PDF):**
 1. **Title Page:** Project Name, Team Members, Student IDs, Chosen Tier.
 2. **Project Proposal:** Chosen SDLC Methodology (e.g., Scrum) and team role breakdown. Explicitly state which student owns which Use Case.
 3. **Problem Statement:** A brief summary of the client's needs.
 4. **Functional Requirements (FRs):** Bulleted list of what the system *shall* do, mapped to your chosen use cases and the 2 required reports. (Use "The system shall..." format).

5. **Quality Attributes (NFRs):** Define specific, measurable non-functional requirements. You must define at least three quality attributes (e.g., *Performance*: "System responds under 2 seconds"; *Security*: "Passwords are encrypted"; *Usability*: "Responsive design for mobile viewing").
- **Presentation 1 (5%):** A 5-7 minute "Solution Pitch" where all members speak and claim ownership of their specific use case.

Part 2: Software Design, Architecture & UI/UX (2 Weeks)

- **Total Weight:** 25% (20% Document & Figma + 5% Presentation)
- **The "1 Use Case Per Student" Requirement:** Students must perform deep UML modeling and UI design specifically for their individually assigned use cases.
- **Document Checklist (What to include in your PDF):**
 1. **Overall Use Case Diagram:** Showing the system boundary and Actor generalizations.
 2. **Use Case Specifications:** For your chosen use cases, write a detailed breakdown including *Use Case Name*, *Actor*, *Pre-conditions*, *Main Flow (Step-by-step)*, *Alternate/Exception Flows*, and *Post-conditions*.
 3. **Behavioral Diagrams:** A UML Sequence Diagram OR Activity Diagram for your specific use cases.
 4. **Structural & CRC Design:** Create **CRC (Class-Responsibility-Collaborator)** cards to outline object interactions, followed by a formal UML Class Diagram (or ERD).
 5. **Architecture & Quality Design:** Visualize your 3-Tier or MVC setup. Briefly explain how your architecture incorporates **Quality Design** principles (e.g., SOLID principles, separation of concerns, high cohesion/low coupling) to satisfy the Quality Attributes defined in Part 1.
 6. **UI/UX Design:** Screenshots and a public link to your **Figma Prototype**. Web tiers must be designed with a responsive grid system in mind (e.g., **Bootstrap** container layouts).
- **Presentation 2 (5%):** A 7-10 minute "Design Review" walking through your CRC cards, database logic, and Figma prototype.

Part 3: Implementation, Testing & Product Demo (3 Weeks)

- **Total Weight:** 35% (30% Code/QA + 5% Presentation)
- **The "1 Use Case Per Student" Requirement:** The codebase and Quality Assurance testing must prove that each student's chosen use case functions perfectly according to the design, ensuring complete CRUD operations.
- **Deliverables Checklist:**
 1. **Source Code:** Complete source code in a .zip file (Ensure **Bootstrap** is utilized for Web Tiers).

2. **Version Control:** A link to your GitHub Repository. *Requirement: >10 meaningful commits per student to prove equitable work.*
3. **Detailed Design & Justification:** A written section explaining your final implemented system design. You must explicitly compare this to your original Part 2 design, justifying any structural, database, or logic changes made during implementation, or explaining why no changes were necessary.
4. **QA & Testing Report (PDF):** * **Test Plan:** A table detailing your Test Cases (Test ID, Description, Input Data, Expected Result, Actual Result, Pass/Fail).
 - **Test Design Techniques:** Explicitly show how you used Equivalence Partitioning (EP) and Boundary Value Analysis (BVA) to test the 15% discount logic.
 - **Execution Evidence:** Screenshots of passed Unit Tests.
 - *(Tier 4 Only):* Source code and execution logs for Automation Tests (e.g., Selenium, Playwright).
5. **Verification of Given Solution:** Include a simple Requirements Traceability Matrix (RTM) or a "Verification Summary" proving that the final software successfully implements the original Functional Requirements and Quality Attributes (NFRs) proposed in Part 1.
6. **Deployment:** A deployed app link (for web) or the executable .exe file path (for desktop).
- **Presentation 3 (5%):** A 10-15 minute live "Final Product Demo." Students must successfully execute their chosen use cases and showcase the 2 reports live on screen.

Final Submission: Comprehensive Report & Video Demo

- **Total Weight:** 20% (10% Report + 10% Video)
- **Deliverables:**
 1. A single, consolidated PDF report of Parts 1, 2, and 3. **Must include PDF bookmarks for easy navigation.** You must apply any feedback the instructor gave you during the previous presentations.
 2. **Lessons Learnt:** A dedicated section at the end of the consolidated report where each team member reflects on their SDLC journey, the challenges they faced during the project, and how they overcame them.
 3. A polished .mp4 Video Demo (or unlisted YouTube link) walking through the working product end-to-end.

5. Evaluation Rubric (100% Scale)

Note: Your final calculated percentage will be capped based on your chosen Technology Tier (e.g., if you score 100% on the rubric but choose Tier 1, your final recorded score is 7.0/10).

Canvas Assignment Item	Weight	Criteria for Full Marks
Part 1: Proposal & Req. Doc	15%	Clearly defines 1 core use case per student (ensuring CRUD) and 2 reports. Comprehensive FRs and measurable Quality Attributes (NFRs) . Professional formatting.
Part 1: Presentation	5%	Clear pitch. All members speak and claim ownership of their specific use case.
Part 2: Design Doc & Figma	20%	Includes Use Case Specs, Sequence/Activity diagrams, CRC Design , and Class diagrams. Demonstrates Quality Design principles. Figma prototype (Bootstrap-ready) maps exactly to these flows and reports.
Part 2: Presentation	5%	Effective technical walkthrough. UI prototype correctly reflects the multi-payment, category requirements, and reports.
Part 3: Implementation & QA	30%	Chosen use cases (with complete CRUD) execute flawlessly. Strict MVC/3-Tier separation and Bootstrap utilized. Thorough Test Cases (EP/BVA). Includes clear Detailed Design Justification and successful Verification of Given Solution (Traceability). <i>(Tier 4 includes Automation Tests)</i> .
Part 3: Live Demo Presentation	5%	Flawless live execution of the chosen use cases and reports. Successfully handles audience Q&A.
Final	10%	Parts 1, 2, and 3 combined into a single, polished PDF

Comprehensive Report		with bookmarks. Includes insightful Lessons Learnt from all members. Previous feedback addressed.
Video Demo Product	10%	High-quality video showcasing the chosen use cases and reports working end-to-end with clear audio/narration.

Declaration of Generative AI Use

The use of Gen AI tools (ChatGPT, Google Gemini, etc.) for brainstorming or refining language is permitted. However, all use must be transparent. You must include an **Appendix.pdf** with a "Generative AI Usage Log" containing: The Tool Used, The Full Prompt, and The Full Output. Failure to declare AI usage will be treated as a breach of academic integrity.

A Note on the Importance of Presentations

While the presentation components (the Pitch, the Demo, and the Final Presentation) each carry a smaller weight than the written reports or prototypes, they are crucial to your overall success in this project. Here is why we emphasize them:

- **Bridging the Gap Between Tech and Business:** In the real world, a brilliant technical solution is only useful if stakeholders understand it. As System Analyst consultants, you must practice translating complex technical work (like relational databases and predictive features) into clear business value (revenue, efficiency, and customer retention) for an executive audience.
- **Proving Authentic Understanding:** With the integration of Generative AI tools and group collaboration, your live presentation and Q&A act as the ultimate validation of your work. You must be able to actively navigate your app prototype and explain *why* you chose specific data models. If you cannot confidently defend what your charts mean for the business, the technical work loses its impact.
- **Guiding the Assessor:** A clear, confident presentation acts as a live executive summary for your grader. By verbally highlighting your cleverest insights and strategic design choices, you frame the assessor's mindset, helping them know exactly what strengths to look for when grading your written documents and dashboards.
- **Synthesizing the Project:** This project is deliberately split into strategy, design, and analytics. The final presentation is the only place where you are forced to tie all three together. It requires you to show how your initial proposal directly influenced your app design, which was then validated by your data analytics.

(Please refer to the separate "FP Student Survival Guide" to ensure you can survive the 7 weeks and, importantly, know what to do after arriving on the island during the first week haha 😊). Good luck. Enjoy your life.